

Charged Particles: Sequential, Parallel and Distributed Execution

Performance comparison of three execution models for an $O(n^2)$ particle simulation

Gregor Antonaz

Univerza na Primorskem

Računalništvo in informatika

Koper, Slovenija

89201088

ABSTRACT

This report compares three execution models of the ChargedParticles simulation: sequential, shared-memory parallel using a thread pool, and distributed using MPI (MPJ Express). All three execute the same force kernel over the same data layout, so the measured differences come only from how the work is divided. On a single ten-core machine the thread pool is the fastest of the three at every problem size, reaching a speedup of $4.8\times$ over the sequential baseline. The distributed model is slower than the thread pool, as expected on one machine, because it copies the complete particle state between processes once per cycle; its disadvantage falls from 278 % at $n = 500$ to 30 % at $n = 3000$ as computation grows faster than communication. Measured runtime is linear in the number of cycles and quadratic in the number of particles, matching the $O(n^2)$ algorithm, and all three models produce identical results.

KEYWORDS

parallel computing, distributed computing, MPI, MPJ Express, N-body simulation, thread pool, speedup, parallel efficiency

1 INTRODUCTION

The simulation models n charged particles moving in a two-dimensional box. Every particle interacts with every other particle through a Coulomb-style force, so each cycle consists of two phases:

- (1) computing the force on every particle from all other particles – $O(n^2)$,
- (2) updating velocities and positions from those forces – $O(n)$.

The first phase dominates the runtime and is the part worth parallelising. Within a single cycle the force on one particle does not depend on the force on any other particle, so the particles can be divided among any number of workers without changing the result.

Three execution models are implemented and compared:

- (1) **Sequential** – one thread computes all particles. This is the baseline.
- (2) **Parallel** – a fixed pool of T threads inside one process, sharing memory.
- (3) **Distributed** – P MPI processes that do not share memory and must exchange state explicitly.

2 IMPLEMENTATION

2.1 Shared kernel and data layout

Comparing execution models is only meaningful if they execute the same algorithm. If the implementations differed in their inner loop

or in how they store particles, the measured ratios would reflect those differences instead of the effect of parallelisation.

All three models therefore call the same kernel class, `ForceKernel`, over the same flat array of doubles:

`[x, y, vx, vy, charge, mass] → 6 doubles per particle`

The kernel offers two operations:

```
computeForces(state, n, start, end, forces, params)
integrate(state, start, end, forces)
```

`computeForces` computes the forces for the particles in the range `[start,end)` against all n particles, and writes them into a globally indexed output array at positions `forces[i*2]` and `forces[i*2+1]`. Because the ranges given to different workers are disjoint, several workers write into the same array without any locking.

2.2 How the three models divide the work

The models differ only in *who* calls the kernel, over *which* range, and how they synchronise at the end of a cycle.

Table 1: The three execution models

Model	Workers	Synchronisation per cycle
Sequential	1 thread	none
Parallel	T threads	invokeAll barrier
Distributed	P processes	Allgatherv collective

Worker k receives the range $[kc, \min((k+1)c, n))$, where $c = \lceil n/T \rceil$ (respectively $\lceil n/P \rceil$) is the chunk size. The sequential model is the special case of a single worker over $[0, n)$. When n is not divisible by the number of workers, the first $n \bmod P$ workers each receive one particle more.

In the parallel model all threads see one shared array. Once the barrier is reached, every thread already sees the updated state and no data is moved at all.

In the distributed model each process owns a private copy of the complete state and updates only its own chunk. At the end of every cycle the chunks must therefore be collected back into a complete state on every process. This is done with a single `Allgatherv` collective, in which every process contributes its own chunk and receives everyone else's:

```
MPI.COMM_WORLD.Allgatherv(
    state, sendOffset, sendCount, MPI.DOUBLE,
    state, 0, recvCounts, displacements, MPI.DOUBLE);
```

Each process therefore moves $6 \cdot 8 \cdot n = 48n$ bytes per cycle. For $n = 2000$ that is 96 kB per process, repeated once for every cycle. This communication cost grows linearly with n , while the computation it enables grows quadratically — a relationship that Section 6.2 returns to.

2.3 Class structure

Table 2: Main classes

Class	Responsibility
ForceKernel	force computation and integration, shared by all models
Simulation	interface: <code>performOneCycle</code> , <code>syncToParticles</code>
AbstractSimulation	shared state: the flat particle array and the force array
SequentialSimulation	one thread over the whole range
ParallelSimulation	thread pool, one task per chunk
DistributedSimulation	MPI processes, <code>Allgatherv</code> after each cycle
SimulationRunner	entry point, shared cycle loop, particle generation
DistributedRunner	MPI entry point
SimulationUI	visualisation and controls

The cycle loop is written once, in `SimulationRunner.runCycles`, and is used by all three models. Particles are generated from a fixed random seed (42), so every model starts from an identical initial state.

3 TEST ENVIRONMENT

- **CPU:** Apple M4, 10 cores — 4 performance cores and 6 efficiency cores
- **Runtime:** OpenJDK 24.0.1
- **MPI:** MPJ Express 0.44, multicore configuration
- **OS:** macOS 26.5.1

MPJ Express is used in its multicore configuration, which runs all MPI processes on this single machine. The distributed model is therefore measured as a *programming model* — explicit partitioning plus collective communication — on one node, rather than across a network. The core layout of 4 performance and 6 efficiency cores matters for interpreting the efficiency figures and is discussed in Section 6.3.

4 METHOD

Two sets of measurements were taken:

- **Test 1** — fixed problem size, varying number of cycles: $n = 2000$, cycles $\in \{250, 500, 1000, 2000\}$.
- **Test 2** — fixed number of cycles, varying problem size: cycles = 1000, $n \in \{500, 1000, 2000, 3000\}$.

Every configuration was measured in four settings: sequential, parallel with 10 threads, MPI with 4 processes and MPI with 10 processes. All runs are headless (`-ui false`) so that only the computation is timed, and each configuration was run repeatedly with

the shortest time reported. Other activity on the machine can only add time to a run, never remove it, so the shortest run is the measurement least affected by interference.

The complete measurement is produced by a single script:

```
./benchmark.sh
```

5 RESULTS

5.1 Test 1: 2000 particles, varying number of cycles

Table 3: Runtime in seconds, $n = 2000$

Cycles	Seq.	Parallel	MPI ($P=4$)	MPI ($P=10$)
250	1.782	0.394	0.543	0.471
500	3.499	0.724	1.048	0.922
1000	6.944	1.453	2.564	2.386
2000	13.952	3.048	5.479	4.737

5.2 Test 2: 1000 cycles, varying number of particles

Table 4: Runtime in seconds, 1000 cycles

Particles	Seq.	Parallel	MPI ($P=4$)	MPI ($P=10$)
500	0.440	0.161	0.252	0.608
1000	1.733	0.467	0.591	0.798
2000	6.944	1.453	2.564	2.386
3000	15.665	4.159	7.716	5.416

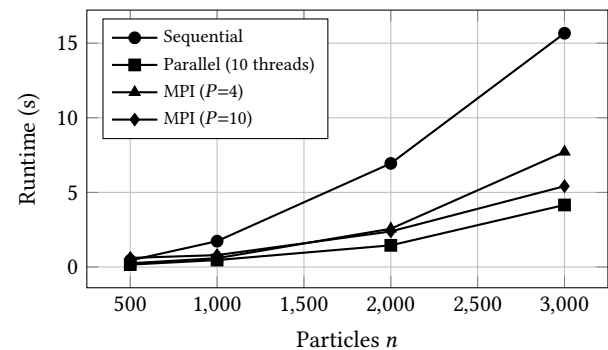


Figure 1: Runtime against particle count at 1000 cycles. All four curves grow quadratically; the parallel and distributed curves grow with a smaller constant factor.

Table 5: Speedup and efficiency at 1000 cycles

n	Parallel (10)		MPI ($P=4$)		MPI ($P=10$)	
	Sp.	Eff.	Sp.	Eff.	Sp.	Eff.
500	2.73	27.3 %	1.75	43.7 %	0.72	7.2 %
1000	3.71	37.1 %	2.93	73.3 %	2.17	21.7 %
2000	4.78	47.8 %	2.71	67.7 %	2.91	29.1 %
3000	3.77	37.7 %	2.03	50.8 %	2.89	28.9 %

5.3 Speedup and efficiency

Speedup is the sequential runtime divided by the runtime of the model, and efficiency is the speedup divided by the number of workers used.

The $n = 500$ row is worth noting: MPI with 10 processes reaches a speedup of 0.72 $\times$, meaning it is *slower than the single-threaded baseline* despite using ten workers. At that size there is so little computation per cycle that the 1000 collective operations dominate the entire run.

6 ANALYSIS

6.1 The measured cost matches the algorithm

Every cycle performs the same amount of work, so runtime should be proportional to the number of cycles. Dividing the Test 1 run-times by the cycle count gives the cost of a single cycle:

Table 6: Cost of one cycle in milliseconds, $n = 2000$

Cycles	Seq.	Parallel	MPI ($P=4$)	MPI ($P=10$)
250	7.13	1.58	2.17	1.88
500	7.00	1.45	2.10	1.84
1000	6.94	1.45	2.56	2.39
2000	6.98	1.52	2.74	2.37

The sequential cost per cycle is constant to within 2.7 % across an eightfold change in run length, and the parallel cost to within 9 %: neither model carries an overhead that grows with the number of cycles. The MPI figures rise somewhat over the same range, so the MPI runs grow a little faster than linearly; with ten processes occupying all ten cores, a longer run is more exposed to whatever else the machine is doing.

The force phase compares every particle with every other particle, so runtime should grow with the square of the particle count. The sequential column of Test 2 confirms this closely:

Table 7: Growth of sequential runtime with problem size

Change in n	Measured factor	Expected ($(n_1/n_0)^2$)
500 $\rightarrow$ 1000	3.94	4.00
1000 $\rightarrow$ 2000	4.01	4.00
2000 $\rightarrow$ 3000	2.26	2.25

All three agree with the prediction to within 1.5 %, which confirms the $O(n^2)$ cost of the pairwise force computation.

6.2 Threads are faster than MPI on a single machine

The thread pool gives the shortest runtime at every problem size measured. This is the expected result, and it follows directly from the difference between the two models.

Both divide exactly the same work among the same number of workers, and both synchronise once per cycle. What differs is the cost of that synchronisation. The threads share one array, so when the barrier is reached every thread already sees the updated state and nothing is copied. The MPI processes each hold a private copy, so every cycle ends with an Allgather that moves $48n$ bytes per process. None of that copying advances the computation; it exists only because the processes cannot see each other's memory.

Table 8: How much slower MPI ($P=10$) is than the thread pool, 1000 cycles

n	Parallel (s)	MPI $P=10$ (s)	MPI slower by
500	0.161	0.608	278 %
1000	0.467	0.798	71 %
2000	1.453	2.386	64 %
3000	4.159	5.416	30 %

The trend is as informative as the values. Communication grows linearly with n while computation grows quadratically, so the share of the runtime spent communicating must fall as the problem grows — and it does, from 278 % down to 30 %. Extrapolating the trend, the two models would meet at a problem size large enough for the $O(n^2)$ work to make the $O(n)$ exchange negligible.

The distributed model pays for a capability that a single machine does not need. Its advantage appears when the processes run on different computers, where the combined memory and the combined core count of several machines become available. Within one machine that advantage is absent, so the communication is pure overhead, and the thread pool winning is exactly what the model predicts.

6.3 More workers is not the same as more efficiency

Four MPI processes use each worker roughly twice as well as ten do: efficiency is 51–73 % at $P = 4$ against 22–29 % at $P = 10$. Ten workers still give the shorter runtime at the larger sizes, but each of them contributes much less.

This follows from the processor layout. The M4 has four performance cores and six efficiency cores. Four workers are scheduled onto the four performance cores, and each contributes close to a full core's worth of throughput. The six further workers land on efficiency cores, which are slower, so each of them adds less than the first four did. Speedup therefore keeps rising while efficiency per worker falls, and the overall speedup saturates well below the $10\times$ that a ten-core count suggests.

Two properties of the implementation reinforce this. The particles are divided into chunks of equal size, and every cycle ends with a barrier. A worker on an efficiency core is given the same amount of work as a worker on a performance core but needs longer to

finish it, and the barrier makes every other worker wait. Sizing chunks in proportion to core speed, or replacing static chunks with dynamic work distribution, would recover part of that loss.

The problem size also decides which configuration wins. At $n = 500$ four processes beat ten (0.252 s against 0.608 s), because with so little work per cycle the cheaper collective matters more than the extra workers. At $n = 3000$ ten processes beat four (5.416 s against 7.716 s), because by then there is enough computation for the additional workers to pay for themselves.

7 CORRECTNESS

All particles are generated from a fixed random seed, so every model starts from an identical initial state and must reach an identical final state. Forces are also summed in the same index order in every model, so no floating-point additions are reordered and the results agree exactly rather than approximately.

The following runs were compared and produced identical particle states:

Table 9: Configurations verified to produce identical output

Model	Workers	Configuration
Sequential	1	$n = 500$, 300 cycles
Parallel	10 threads	$n = 500$, 300 cycles
Distributed	2, 4 and 7 processes	$n = 500$, 300 cycles

The particle count 500 is deliberately not divisible by 7. The distributed implementation gives the first $n \bmod P$ processes one particle more than the rest, and this case exercises that uneven division.

8 REPRODUCING THE MEASUREMENTS

MPJ Express is bundled with the project in `lib/mpj`, so no separate installation is needed. From the project directory:

```
mvn compile      # build
./benchmark.sh   # produces every table in this report
```

A single configuration can also be run directly:

```
./run_sequential.sh --particles 2000 \
  --cycles 1000 --ui false
./run_parallel.sh --particles 2000 \
  --cycles 1000 --ui false
./run_distributed.sh 4 --particles 2000 \
  --cycles 1000 --ui false
```

Each run prints the number of particles, the number of cycles, the elapsed time and the state of the first five particles; running the three commands above and comparing those five lines is the correctness check of the previous section. Omitting `--ui false` opens the visualisation instead, and in the distributed case the window is opened by the process with rank 0. All three models can also be selected from the graphical interface.

9 CONCLUSIONS

- (1) **Both parallel models give a substantial speedup.** Against the sequential baseline the thread pool reaches 4.8× and MPI reaches 2.9×, on a machine with four performance cores.
- (2) **Threads beat MPI on a single machine.** With an identical kernel and an identical division of work, the thread pool is 30–278 % faster than MPI with the same number of workers, because MPI must copy the complete particle state between processes once per cycle while threads share it directly.
- (3) **The distributed disadvantage shrinks as the problem grows.** Communication is $O(n)$ while computation is $O(n^2)$, so the MPI penalty falls from 278 % at $n = 500$ to 30 % at $n = 3000$.
- (4) **More workers is not the same as more efficiency.** Four processes reach 51–73 % efficiency against 22–29 % for ten, because only four of the ten cores are performance cores.
- (5) **The measured cost matches the algorithm.** Runtime is linear in the number of cycles and quadratic in the number of particles, as expected for $O(n^2)$ pairwise force computation.
- (6) **All three models are numerically identical.** Starting from the same seed, the sequential, parallel and distributed runs produce exactly the same particle states.